

Ai percentage:

Ai percentage:

HandleUserInput function, some cases were Ai generated.

ParseInput function was ai generated

Recommend function was ai generated

updatepreference

Summarizeconversation function was ai generated

Getusermood was ai generated

Mostdiscussedtopics was ai generated

6.5/16 functions were Ai

40% Ai

Imperative → Declarative

getUserMood inside memory.scala, Ai-Generated imperative code:

getUserMood — imperative C++:

```
cpp

string getUserMood(vector<InteractionEntry> history) {
    int positiveCount = 0;
    int negativeCount = 0;
    vector<string> positiveWords = {"great", "love", "amazing", "good", "nice"};
    vector<string> negativeWords = {"boring", "bad", "hate", "don't like", "awful"};
    for (int i = 0; i < history.size(); i++) {
        string input = history[i].userInput;
        for (int j = 0; j < positiveWords.size(); j++) {
            if (input.find(positiveWords[j]) != string::npos) {
                positiveCount++;
            }
        }
        for (int k = 0; k < negativeWords.size(); k++) {
            if (input.find(negativeWords[k]) != string::npos) {
                negativeCount++;
            }
        }
    }
    if (positiveCount > negativeCount) return "positive";
    else if (negativeCount > positiveCount) return "negative";
    else return "neutral";
}
```

Converted written declarative version:

```
def getMostDiscussedTopics(history : List[InteractionEntry]) : List[(String,Int)] = {
    history.groupBy(x => x.intent).mapValues(_.size).toList.sortBy(x => (x._2,x._1)).reverse
}

def getUserMood(history : List[InteractionEntry]) : String = {
    val positive = List("great", "love", "amazing", "good", "nice")
    val negative = List("boring", "bad", "hate", "don't like", "awful")
    val pair = history.foldLeft((0,0))((acc,x) => (
        if (positive.filter(y => x.userInput.contains(y)).nonEmpty) (acc._1 + 1) else (acc._1),
        if(negative.filter(y => x.userInput.contains(y)).nonEmpty) (acc._2 + 1) else (acc._2)
    ))
    val mood = pair match {
        case (a,b) if (a > b) => "positive"
        case (a,b) if (a < b) => "negative"
        case _ => "neutral"
    }
    mood
}
```

summarizeConversation inside memory.scala, Ai-Generated imperative version of the code:

```
cpp
string summarizeConversation(vector<InteractionEntry> history) {
    int totalInteractions = history.size();
    int recommendationCount = 0;
    int greetingCount = 0;

    for (int i = 0; i < history.size(); i++) {
        if (history[i].intent == "recommendation") {
            recommendationCount++;
        } else if (history[i].intent == "greeting") {
            greetingCount++;
        }
    }

    string summary = "We've had " + to_string(totalInteractions) + " exchanges. ";

    if (recommendationCount > 0) {
        summary += "You asked for " + to_string(recommendationCount) + " recommendat
    }
    if (greetingCount > 0) {
        summary += "We exchanged " + to_string(greetingCount) + " greetings. ";
    }

    return summary;
}
```

Written declarative version in the submitted code file:

```
def summarizeConversation(history : List[InteractionEntry]): String = {
    val recommendationCounts = history.filter(_.intent == "recommendation").size
    val greetingCounts = history.filter(_.intent == "greeting").size
    val summary = "We've had " + history.size.toBinaryString + " exchanges."

    val summaryPart1 = recommendationCounts match
        case recommendationCounts if (recommendationCounts > 0) => "You asked for " + recommendationCounts.toBinaryString + " recommendations. "
        case _ => summary

    val summaryPart2 = greetingCounts match
        case greetingCounts if (greetingCounts > 0) => "We exchanged " + greetingCounts.toBinaryString + " greetings. "
        case _ => summary

    val finalSummary = summary + summaryPart1 + summaryPart2
    finalSummary
}
```

```
def parseInput(input: String): List[String] = {
  input.toLowerCase
    .replaceAll("[^a-zA-Z0-9\\s]", "")
    .split("\\s+")
    .toList
}
```

```
def handleUserInput(input: String, memory : ConversationState): String = {
  input.toLowerCase match {

    case x if (x == "hello") || (x == "hi") || x.startsWith("hello") || x.startsWith("hi") => greetUser() // case to greet the user

    case x if x.contains("protein") || x.contains("carbs") || x.contains("fat") || x.contains("calories") => generateResponse(x) // case for topic queries

    case x if x.contains("recipe") || x.contains("meal") || x.contains("dish") || x.contains("food") || x.contains("eat") ||
      x.contains("suggest something") || x.contains("what do you recommend") || x.contains("what do you recommend") =>
      val prefs = RecommendationEngine.getUserPreferences(memory)
      val answer = RecommendationEngine.recommend(prefs, data.allRecipes)
      if (answer.isEmpty) "I couldn't find recipes matching your preferences. Try updating them!"
      else answer.map(RecommendationEngine.explainRecommendation).mkString("\n")

    case x if x.contains("vegan") || x.contains("prefer") || x.contains("vegetarian") ||
      x.contains("gluten-free") || x.contains("dairy-free") || x.contains("nut-free") || x.contains("high-protein") || x.contains("low-carb")
      => RecommendationEngine.updatePreferences(x, memory) //case for preference updates

    case x if x.contains("summarize") || x.contains("summary") || x.contains("what have we talked") => conversationMemory.summarizeConversation(memory.history)

    case x if x.contains("topics") || x.contains("what topics") => val topics = conversationMemory.extractTopics(memory.history, List())
      "We've discussed: " + topics.mkString(", ")

    case x if x.contains("most discussed") || x.contains("popular") => {
      val topics = conversationMemory.getMostDiscussedTopics(memory.history)
      topics.map(t => s"${t._1}: ${t._2} times").mkString("\n")
    }

    case x if x.contains("recent") || x.contains("last") => {val last = conversationMemory.getLastNInteractions(3, memory).map(e => s"You: ${e.userInput}\nMe: ${e.botResponse}")
      last.mkString("\n---\n")
    }

    case _ => "I'm sorry, I didn't understand that. Can you please rephrase your request?" // case for fallbacks
  }
}
```

4 cases were Ai

```

def recommend(preferences: Map[String, String], recipes: List[Recipe]): List[Recipe] = {

  recipes.filter(recipe => preferences.get("cuisine").forall(prefCuisine => recipe.cuisine.equalsIgnoreCase(prefCuisine)
    )

    &&

    preferences.get("difficulty")
      .forall(prefDifficulty =>
        recipe.difficulty.equalsIgnoreCase(prefDifficulty)
      )

    &&

    preferences.get("diet")
      .forall(prefDiet =>
        recipe.dietaryTags.exists(tag =>
          tag.equalsIgnoreCase(prefDiet)
        )
      )
  )

  .sortBy(_.prepTime)
}

```

```

def updatePreferences(input: String, memory: ConversationState): String = {
  val key = "diet"
  val value =
    if (input.contains("vegan")) "vegan"
    else if (input.contains("vegetarian")) "vegetarian"
    else if (input.contains("gluten-free")) "gluten-free"
    else "none"
  s"Got it! I'll remember you prefer $value food."
}

```

Direct AI Usage

1. DataStructures.scala

- Ai generated knowledge base data : `val allRecipes: List[Recipe]`, i defined Recipe case class myself

2. Memory.scala

- Ai generated `val timestamp = java.time.LocalDateTime.now().toString` inside `logInteraction(userInput : String, botResponse : String, context : ConversationState) : ConversationState`

3. Chatbot.Scala

- Ai generated function `parseInput(Input: String)`
- Ai generated some of the query topics and preferences for extra ideas
- Ai modified the message output for the topic queries, for example `Protein is essential for building and repairing tissues. It can be found in foods like meat, dairy, legumes, and nuts.` If the user asks about protein

4. Recommendation Engine: 6 functions, my branch : feature/recommendation-engine

1. Improving logic with `forall` and `exists`

My original implementation had logical inefficiencies in certain checks. Through AI-assisted analysis, I learned that using higher-order functions like `forall` and `exists` provides a more concise and efficient way to express these conditions, improving both readability and performance.

2. Returning a Map in `updatePreferences`

The `updatePreferences` function returns a `Map` because the updated state must be explicitly captured. If the return value is not stored, the changes are lost. This design avoids unintended side effects and enforces functional programming principles by ensuring immutability and explicit state updates.

3. Correct ordering in `getTopRecommendations` transformation pipeline

I was initially unsure whether to apply `.take(3)` before or after `.sortBy(_.prepTime)`. I learned that order matters significantly: applying `.take(3)` first would select arbitrary items before sorting, leading to incorrect results. Instead, sorting first ensures that the fastest recipes across the entire dataset are considered before limiting the output to the top three.

4. Using `Option[String]` instead of `null` with `Map.get`

I used `preferences.get("cuisine")`, which returns an `Option[String]`. I chose to work with `Option` instead of `null` because it enforces safe handling of missing values at compile time. This prevents runtime errors such as `NullPointerException` and encourages explicit handling of cases where a value may be absent.